

JomCode Coding Bootcamp

Loops and iteration

Understand, predict, type, run, then practice.

What you will be able to do

- 01 Use for, range and while for distinct jobs.
- 02 Accumulate results and stop iteration intentionally.
- 03 Diagnose off-by-one and nonterminating loops.

Plain meaning → technical view → use

PLAIN MEANING

Repeat a task without copying the same instruction many times.

TECHNICAL VIEW

for, range and while iterate over data or continue until a condition changes.

WHEN YOU USE IT

Count registrations and inspect workshop records.

TIP

Trace the loop variable and total on paper for two iterations first.

Worked example 1: Numbered attempts



chapter_06_loops_and_iteration.py

```
1 for attempt in range(1, 4):  
2     print(f"Attempt {attempt}")
```

PREDICT Why is the stop value 4?

Example 1: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
Attempt 1  
Attempt 2  
Attempt 3
```

WHY THIS HAPPENS

The stop is exclusive, so three attempts use indexes 1, 2 and 3.

Exercise 1: Running total

BUILD

Sum counts [2, 0, 4] with an explicit loop.

CHECK

Nonempty => 6. Empty => 0. Keep initialization outside the loop.

IF STUCK

Start from the additive identity 0.

Worked example 2: Accumulating seats



chapter_06_loops_and_iteration.py

```
1 total = 0
2 for count in [3, 5, 2]:
3     total += count
4 print(total)
```

PREDICT Where should total be initialized?

Example 2: output and reason

TERMINAL	>_	EXPECTED OUTPUT
----------	----	-----------------

10		
----	--	--

WHY THIS HAPPENS

Initialization outside the loop preserves previous additions.

Exercise 2: Valid capacities

BUILD

Print only capacities from 1 to 5 from [0, 1, 5, 6].

CHECK

Printed values are 1 then 5. Both boundaries are inclusive.

IF STUCK

Combine a loop and one boundary condition.

Worked example 3: Bounded while



chapter_06_loops_and_iteration.py

```
1 attempt = 0
2 while attempt < 3:
3     attempt += 1
4     print(attempt)
5 print("Stopped")
```

PREDICT What happens if attempt += 1 disappears?

Example 3: output and reason

TERMINAL	>_	EXPECTED OUTPUT
----------	----	-----------------

1		
2		
3		
Stopped		

WHY THIS HAPPENS

The condition stays true forever. Interrupt with Ctrl+C, then restore progress.

Exercise 3: Countdown repair

BUILD

Write a countdown from 3 to 1 that terminates and then prints Done.

CHECK

3,2,1,Done. Starting from 0 prints only Done.

IF STUCK

Decrease the same value used by the condition.

Checkpoint

PROVE IT

Write a terminating loop and trace every state update before execution.